

In **Java 8+ Stream API**, the `filter()` method is used to **select elements based on a boolean condition**.

✔ 1 Method Signature of `filter()`

```
Stream<T> filter(Predicate<? super T> predicate)
```

❑ Accepted Input Parameter

`filter()` accepts **only ONE parameter type**:

Parameter Type	Package	Functional Method	Returns
Predicate<T>	java.util.function	boolean test(T t)	true / false

❑ So technically, **`filter()` accepts only a Predicate (boolean condition)**.

✔ 2 What is a Predicate?

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

It takes **one input** and returns **boolean**.

✔ 3 All Possible Ways to Pass Boolean Checks to `filter()`

Although `filter()` only accepts `Predicate`, you can pass it in different forms:

1 Lambda Expression (Most Common)

```
import java.util.*;

public class FilterExample1 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 15, 20, 25);

        numbers.stream()
```

```

        .filter(n -> n % 2 == 0)                // boolean condition
        .forEach(System.out::println);
    }
}

```

✓ `n -> n % 2 == 0` returns boolean.

2 Method Reference

```

import java.util.*;
import java.util.function.Predicate;

public class FilterExample2 {

    public static boolean isEven(Integer n) {
        return n % 2 == 0;
    }

    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 15, 20, 25);

        numbers.stream()
            .filter(FilterExample2::isEven)
            .forEach(System.out::println);
    }
}

```

✓ Method must return boolean.

3 Anonymous Class

```

import java.util.*;
import java.util.function.Predicate;

public class FilterExample3 {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(10, 15, 20, 25);

        numbers.stream()
            .filter(new Predicate<Integer>() {
                @Override
                public boolean test(Integer n) {
                    return n % 2 == 0;
                }
            })
    }
}

```

```
        }  
    })  
    .forEach(System.out::println);  
}  
}
```

✓ Old Java style (Java 8 compatible).

4 Predicate Variable

```
import java.util.*;  
import java.util.function.Predicate;  
  
public class FilterExample4 {  
    public static void main(String[] args) {  
  
        List<Integer> numbers = Arrays.asList(10, 15, 20, 25);  
  
        Predicate<Integer> evenCheck = n -> n % 2 == 0;  
  
        numbers.stream()  
            .filter(evenCheck)  
            .forEach(System.out::println);  
    }  
}
```

5 Multiple Boolean Conditions

Using && (AND)

```
.filter(n -> n > 10 && n % 2 == 0)
```

Using || (OR)

```
.filter(n -> n > 20 || n % 2 == 0)
```

6 Using Predicate Default Methods

Predicate provides:

```
and()  
or()  
negate()
```

Example:

```
Predicate<Integer> greaterThan10 = n -> n > 10;  
Predicate<Integer> even = n -> n % 2 == 0;
```

```
numbers.stream()  
    .filter(greaterThan10.and(even))  
    .forEach(System.out::println);
```

✔ 4 Special Primitive Stream Filters

Java provides specialized stream types:

Stream Type	filter() accepts
IntStream	IntPredicate
LongStream	LongPredicate
DoubleStream	DoublePredicate

Example with IntStream

```
import java.util.stream.IntStream;  
  
public class IntStreamFilterExample {  
    public static void main(String[] args) {  
  
        IntStream.range(1, 10)  
            .filter(n -> n % 2 == 0)  
            .forEach(System.out::println);  
    }  
}
```

✔ Uses IntPredicate

✔ 5 Complete List of Accepted Functional Interfaces

Stream Type	Parameter Type
Stream<T>	Predicate<T>
IntStream	IntPredicate

LongStream	LongPredicate
DoubleStream	DoublePredicate

☐ No other functional interfaces are accepted directly.

☐ Interview-Ready Answer

The `filter()` method in Java Stream API accepts a single argument of type `Predicate<T>` (or primitive variants like `IntPredicate`). The predicate must return a boolean value. It can be passed using a lambda expression, method reference, anonymous class, or a `Predicate` variable. Internally, each stream element is passed to the `test()` method, and only elements returning true are retained.

If you want, I can next explain:

- ☐ Internal execution flow of `filter()`
- ☒ Bytecode-level explanation
- ☐ How filter works in parallel stream
- ☐ UML execution diagram
- ☐ Java 1.5 equivalent (without streams)

In Java Stream API, `filter()` is commonly used for **null checks** to remove null elements from a stream.

✔ 1 Method Signature

`Stream<T> filter(Predicate<? super T> predicate)`

So again, **filter()** accepts only a **Predicate**, but here we focus specifically on **null-check predicates**.

✔ 2 All Possible Null-Check Inputs Accepted by `filter()`

Since `filter()` accepts a `Predicate`, null checks can be written in multiple ways.

1 Lambda Null Check (Most Common)

```
import java.util.*;

public class NullFilterExample1 {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", null, "Stream", null);

        names.stream()
            .filter(s -> s != null)
            .forEach(System.out::println);
    }
}
```

✓ Removes null values.

2 Using Objects::nonNull (Recommended)

From **Oracle Corporation Java 8**, the Objects class provides:

```
public static boolean nonNull(Object obj)
```

Example:

```
import java.util.*;
import java.util.stream.*;
import java.util.Objects;

public class NullFilterExample2 {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", null, "Stream", null);

        names.stream()
            .filter(Objects::nonNull)
            .forEach(System.out::println);
    }
}
```

✓ Cleaner and interview-preferred.

3 Using Objects::isNull (To Keep Only Nulls)

```
names.stream()
    .filter(Objects::isNull)
```

```
.forEach(System.out::println);
```

✓ Keeps only null values.

4 Anonymous Class Null Check

```
import java.util.*;
import java.util.function.Predicate;

public class NullFilterExample3 {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", null, "Stream", null);

        names.stream()
            .filter(new Predicate<String>() {
                @Override
                public boolean test(String s) {
                    return s != null;
                }
            })
            .forEach(System.out::println);
    }
}
```

5 Predicate Variable

```
import java.util.*;
import java.util.function.Predicate;

public class NullFilterExample4 {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", null, "Stream", null);

        Predicate<String> notNull = s -> s != null;

        names.stream()
            .filter(notNull)
            .forEach(System.out::println);
    }
}
```

6 Combined Null + Condition Check

```
names.stream()
    .filter(s -> s != null && s.length() > 4)
```

```
.forEach(System.out::println);
```

✓ Prevents NullPointerException.

✓ 3 Primitive Streams and Null

□ Important:

Primitive streams cannot contain null.

Stream Type	Null Allowed?
Stream<T>	✓ Yes
IntStream	✗ No
LongStream	✗ No
DoubleStream	✗ No

Example (invalid scenario):

```
IntStream.of(1, 2, null); // Compile-time error
```

✓ 4 Complete List of Null-Check Inputs Accepted

Technically filter() accepts only:

Stream Type	Parameter
Stream<T>	Predicate<T>
IntStream	IntPredicate
LongStream	LongPredicate
DoubleStream	DoublePredicate

For null checks specifically, valid predicate forms are:

1. `x -> x != null`
2. `Objects::nonNull`
3. `Objects::isNull`

4. Custom Predicate
5. Anonymous class

☐ Interview-Ready Answer

The `filter()` method in Java Stream API accepts a single `Predicate<T>`. For null checks, common inputs are `x -> x != null` and `Objects::nonNull`. Primitive streams do not allow null values. Internally, each element is passed to `Predicate.test()`, and only elements returning true are retained.

If you want next:

- ☐ Internal working of null filtering
- ⚙ Bytecode-level explanation
- ☐ Java 1.5 equivalent null filtering
- ☐ UML execution flow diagram
- ☐ Parallel stream null filtering behavior

In Java Stream API, `filter()` is used for **comparison operations** like `>`, `<`, `==`, `equals()`, date comparisons, string comparisons, object field comparisons, etc.

✔ 1 Method Signature

`Stream<T> filter(Predicate<? super T> predicate)`

So again, **`filter()` accepts only ONE input parameter:**

Stream Type	Accepted Parameter
Stream<T>	<code>Predicate<T></code>
IntStream	<code>IntPredicate</code>
LongStream	<code>LongPredicate</code>
DoubleStream	<code>DoublePredicate</code>

Now let's list **all comparison types** you can perform inside the predicate.

✓ 2 Numeric Comparisons

1 Greater Than (>)

```
import java.util.*;

public class GreaterThanExample {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 10, 15, 20);

        list.stream()
            .filter(n -> n > 10)
            .forEach(System.out::println);
    }
}
```

2 Less Than (<)

```
.filter(n -> n < 15)
```

3 Greater Than or Equal (>=)

```
.filter(n -> n >= 10)
```

4 Less Than or Equal (<=)

```
.filter(n -> n <= 20)
```

5 Equal To (==) (Primitive)

```
.filter(n -> n == 10)
```

⚠ For wrapper types prefer `.equals()`.

✓ 3 Object Comparisons

6 Using `equals()`

```
import java.util.*;

public class EqualsExample {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Java", "Python", "Java");

        names.stream()
            .filter(s -> s.equals("Java"))
            .forEach(System.out::println);
    }
}
```

7 Using equalsIgnoreCase()

```
.filter(s -> s.equalsIgnoreCase("java"))
```

8 Using compareTo()

```
.filter(s -> s.compareTo("M") > 0)
```

✔ 4 String Comparisons

9 startsWith()

```
.filter(s -> s.startsWith("J"))
```

10 endsWith()

```
.filter(s -> s.endsWith("a"))
```

11 contains()

```
.filter(s -> s.contains("va"))
```

12 length comparison

```
.filter(s -> s.length() > 4)
```

✔ 5 Date Comparisons (Java 8 Time API)

From **Oracle Corporation Java 8**, we use LocalDate.

```
import java.time.LocalDate;
import java.util.*;

public class DateComparisonExample {
    public static void main(String[] args) {
        List<LocalDate> dates = Arrays.asList(
            LocalDate.of(2023,1,1),
            LocalDate.of(2025,1,1)
        );

        dates.stream()
            .filter(d -> d.isAfter(LocalDate.now()))
            .forEach(System.out::println);
    }
}
```

Available date comparisons:

- isAfter()
- isBefore()
- isEqual()

✔ 6 Custom Object Field Comparisons

Example: Employee salary comparison

```
import java.util.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }
}

public class ObjectComparisonExample {
```

```
public static void main(String[] args) {  
  
    List<Employee> employees = Arrays.asList(  
        new Employee("A", 30000),  
        new Employee("B", 60000)  
    );  
  
    employees.stream()  
        .filter(e -> e.getSalary() > 50000)  
        .forEach(e -> System.out.println(e.name));  
}  
}
```

✔ 7 Multiple Comparisons

Using AND (&&)

```
.filter(n -> n > 10 && n < 20)
```

Using OR (||)

```
.filter(n -> n == 5 || n == 20)
```

Using Predicate methods

```
Predicate<Integer> greaterThan10 = n -> n > 10;  
Predicate<Integer> lessThan20 = n -> n < 20;
```

```
list.stream()  
    .filter(greaterThan10.and(lessThan20))  
    .forEach(System.out::println);
```

✔ 8 Primitive Stream Comparisons

IntStream Example

```
import java.util.stream.IntStream;  
  
public class IntStreamExample {  
    public static void main(String[] args) {  
        IntStream.range(1, 10)
```

```
        .filter(n -> n > 5)
        .forEach(System.out::println);
    }
}
```

Uses IntPredicate.

✔ 9 Complete List of Comparison Types You Can Use

Inside filter() you can use:

Numeric

- >
- <
- >=
- <=
- ==
- !=

Object

- equals()
- equalsIgnoreCase()
- compareTo()

String

- startsWith()
- endsWith()
- contains()
- length()

Date

- isAfter()
- isBefore()
- isEqual()

Logical

- &&
- ||
- !
- Predicate.and()
- Predicate.or()
- Predicate.negate()

❑ Interview-Ready Answer

The `filter()` method accepts a `Predicate<T>` that returns a boolean. All comparison operations such as numeric comparisons (`>`, `<`), object comparisons (`equals`, `compareTo`), string methods (`startsWith`, `contains`), date comparisons (`isAfter`, `isBefore`), and logical combinations (`&&`, `||`) can be used inside the predicate. Primitive streams use specialized predicates like `IntPredicate`.

If you want next:

- ❑ Internal execution flow of comparisons
- ⚙ Bytecode-level explanation
- ❑ Parallel stream comparison behavior
- ❑ UML execution diagram
- ❑ Java 1.5 equivalent comparison filtering

✔ Java Stream API `filter()` — Method Calls That Return boolean

❑ 1 Official Method Signature

`Stream<T> filter(Predicate<? super T> predicate)`

❑ `filter()` accepts only ONE parameter type:

Stream Type	Accepted Parameter
Stream<T>	<code>Predicate<T></code>
IntStream	<code>IntPredicate</code>
LongStream	<code>LongPredicate</code>
DoubleStream	<code>DoublePredicate</code>

A `Predicate<T>` is:

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

✓ Therefore, **any method call that returns boolean** can be passed to `filter()`.

✓ 2 All Categories of Boolean-Returning Method Calls

1 String Boolean Methods

```
import java.util.*;

public class StringBooleanExample {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("Java", "", "Stream", "JAVA");

        list.stream()
            .filter(s -> !s.isEmpty())
            .filter(s -> s.startsWith("J"))
            .filter(s -> s.equalsIgnoreCase("java"))
            .forEach(System.out::println);
    }
}
```

Common String boolean methods:

- `isEmpty()`
 - `equals()`
 - `equalsIgnoreCase()`
 - `startsWith()`
 - `endsWith()`
 - `contains()`
 - `matches()`
-

2 Objects Utility Boolean Methods

From **Oracle Corporation Java 8:

```
import java.util.*;
import java.util.Objects;

public class ObjectsBooleanExample {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("Java", null, "Stream");

        list.stream()
```

```
        .filter(Objects::nonNull)
        .forEach(System.out::println);
    }
}
```

Boolean methods:

- Objects.nonNull()
 - Objects.isNull()
-

3 Collection Boolean Methods

```
import java.util.*;

public class CollectionBooleanExample {
    public static void main(String[] args) {
        List<List<Integer>> data = Arrays.asList(
            Arrays.asList(1,2),
            Collections.emptyList()
        );

        data.stream()
            .filter(list -> !list.isEmpty())
            .forEach(System.out::println);
    }
}
```

Common methods:

- isEmpty()
 - contains()
 - containsAll()
-

4 Numeric Wrapper Boolean Methods

```
import java.util.*;

public class WrapperBooleanExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 20, 30);

        numbers.stream()
            .filter(n -> n.equals(20))
            .forEach(System.out::println);
    }
}
```

```
}
```

Methods:

- equals()
- compareTo() > 0
- intValue() > 10

5 Date & Time Boolean Methods

```
import java.time.LocalDate;
import java.util.*;

public class DateBooleanExample {
    public static void main(String[] args) {
        List<LocalDate> dates = Arrays.asList(
            LocalDate.now().minusDays(1),
            LocalDate.now().plusDays(1)
        );

        dates.stream()
            .filter(d -> d.isAfter(LocalDate.now()))
            .forEach(System.out::println);
    }
}
```

Common boolean methods:

- isAfter()
- isBefore()
- isEqual()

6 Custom Object Boolean Methods

```
import java.util.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}
```

```

    public boolean isHighSalary() {
        return salary > 50000;
    }
}

public class CustomBooleanExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("A", 30000),
            new Employee("B", 60000)
        );

        employees.stream()
            .filter(Employee::isHighSalary)
            .forEach(e -> System.out.println(e.name));
    }
}

```

✓ Any custom method returning boolean is valid.

7 Predicate Default Boolean Methods

Predicate itself provides boolean-returning combinators:

```

Predicate<Integer> greaterThan10 = n -> n > 10;
Predicate<Integer> even = n -> n % 2 == 0;

```

```

numbers.stream()
    .filter(greaterThan10.and(even))
    .forEach(System.out::println);

```

Methods:

- `and()`
 - `or()`
 - `negate()`
-

8 Primitive Stream Boolean Methods

IntStream Example

```

import java.util.stream.IntStream;

public class IntStreamBooleanExample {

```

```

public static void main(String[] args) {
    IntStream.range(1, 10)
        .filter(n -> isEven(n))
        .forEach(System.out::println);
}

static boolean isEven(int n) {
    return n % 2 == 0;
}
}

```

Uses:

- IntPredicate
- LongPredicate
- DoublePredicate

✓ 3 Complete List of Accepted Boolean-Returning Inputs

filter() accepts:

✓ Lambda returning boolean

`x -> x > 10`

✓ Method reference returning boolean

`ClassName::methodReturningBoolean`

✓ Anonymous class

`new Predicate<T>() { public boolean test(T t) {...} }`

✓ Predicate variable

`Predicate<T> p = ...`

✓ Primitive predicate types

- IntPredicate
- LongPredicate
- DoublePredicate

❑ Interview-Ready Answer

The `filter()` method in Java Stream API accepts a single parameter of type `Predicate<T>` (or primitive variants). Any lambda, method reference, anonymous class, or custom method that returns a boolean value can be passed to `filter()`. Internally, each stream element is evaluated using `Predicate.test()`, and only elements returning true are retained.

If you'd like next:

- ☐ Internal working of `filter()` step-by-step
- ⚙️ Bytecode-level explanation
- ☐ Parallel stream behavior
- ☐ UML execution flow diagram
- ☐ Java 1.5 equivalent implementation

Java Stream API `distinct()` — How It Works (`equals()` / `hashCode()`)

❑ 1 Official Method Signature

`Stream<T> distinct()`

❑ Important:

- `distinct()` takes **NO input parameters**
 - It is an **intermediate stateful operation**
 - It relies internally on:
 - `equals()`
 - `hashCode()`
-

✓ 2 How `distinct()` Works Internally

For `Stream<T>`:

- Internally uses a **HashSet**
- `HashSet` uses:
 - `hashCode()` → to find bucket
 - `equals()` → to check duplicates

So for custom objects:

- You **must override both** `equals()` and `hashCode()`
-

✔ 3 Example 1 — Primitive Wrapper (Integer)

```
import java.util.*;

public class DistinctExample1 {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(10, 20, 10, 30, 20);

        numbers.stream()
            .distinct()
            .forEach(System.out::println);
    }
}
```

✔ Output:

```
10
20
30
```

Why?

- Integer already overrides `equals()` and `hashCode()`.
-

✔ 4 Example 2 — String (Already Overrides equals & hashCode)

```
import java.util.*;

public class DistinctExample2 {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("Java", "Python", "Java");

        names.stream()
            .distinct()
            .forEach(System.out::println);
    }
}
```

✔ Works because String overrides both methods.

✔ 5 Example 3 — Custom Object WITHOUT equals()/hashCode()

```
import java.util.*;

class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class DistinctExample3 {
    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee(1, "A"),
            new Employee(1, "A")
        );

        list.stream()
            .distinct()
            .forEach(e -> System.out.println(e.id));
    }
}
```

✗ Output:

```
1
1
```

Why?

- Default equals() from Object
- Different memory addresses
- So both objects are considered different

✔ 6 Example 4 — Custom Object WITH equals() and hashCode()

```
import java.util.*;
```

```
class Employee {
    int id;
```

```

String name;

Employee(int id, String name) {
    this.id = id;
    this.name = name;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee)) return false;
    Employee e = (Employee) o;
    return id == e.id;
}

@Override
public int hashCode() {
    return Objects.hash(id);
}
}

public class DistinctExample4 {
    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee(1, "A"),
            new Employee(1, "A"),
            new Employee(2, "B")
        );

        list.stream()
            .distinct()
            .forEach(e -> System.out.println(e.id));
    }
}

```

✓ Output:

1
2

Now duplicates are removed correctly.

✓ 7 Primitive Streams Behavior

Stream Type	Behavior
-------------	----------

IntStream	Removes duplicate primitive values
LongStream	Same
DoubleStream	Same

Example:

```
import java.util.stream.IntStream;

public class IntDistinctExample {
    public static void main(String[] args) {

        IntStream.of(1,2,2,3,3,4)
            .distinct()
            .forEach(System.out::println);
    }
}
```

✓ Works directly (no equals/hashCode needed).

✓ 8 Internal Implementation (Conceptual)

Internally behaves similar to:

```
Set<T> seen = new HashSet<>();
for (T element : streamData) {
    if (seen.add(element)) {
        output(element);
    }
}
```

Since HashSet uses:

- hashCode() first
- equals() second

That's why both are required.

✓ 9 Complete Summary Table

Scenario	equals() Needed	hashCode() Needed
Integer	Already	Already implemented

	implemented	
String	Already implemented	Already implemented
Custom Object	YES	YES
IntStream	No	No

❑ Interview-Ready Answer

The `distinct()` method takes no parameters. It internally uses a `HashSet` to remove duplicate elements. Therefore, it relies on both `equals()` and `hashCode()`. For primitive streams, duplicate removal is based on value comparison. For custom objects, overriding both methods is mandatory to ensure correct duplicate elimination.

If you'd like next:

- ❑ Internal working of `distinct()` step-by-step
- ⚙ Bytecode-level explanation
- ❑ Parallel stream distinct behavior
- ❑ UML execution diagram
- ❑ Java 1.5 equivalent implementation

✔ Java Stream API `distinct()` with Immutable Objects

❑ 1 Method Signature

`Stream<T> distinct()`

❑ Important:

- `distinct()` takes **NO input parameters**
- It relies on:
 - `equals()`
 - `hashCode()`
- Works best with **immutable objects**

✔ 2 Why Immutable Objects Work Perfectly with `distinct()`

An **immutable object**:

- Fields are final
- No setters
- State cannot change after creation
- equals() and hashCode() remain stable

Because distinct() internally behaves like a HashSet, object state **must not change** after being added.

✓ 3 Example 1 — Built-in Immutable Objects

String (Immutable)

```
import java.util.*;

public class ImmutableDistinctExample1 {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Python", "Java");

        list.stream()
            .distinct()
            .forEach(System.out::println);
    }
}
```

✓ Works because String is immutable and overrides equals() and hashCode().

Integer (Immutable)

```
import java.util.*;

public class ImmutableDistinctExample2 {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(10, 20, 10, 30);

        list.stream()
            .distinct()
            .forEach(System.out::println);
    }
}
```

✓ Integer is immutable.

✓ 4 Example 2 — Custom Immutable Object

Step 1 Create Immutable Class

```
import java.util.Objects;

final class Employee {

    private final int id;
    private final String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() { return id; }
    public String getName() { return name; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee)) return false;
        Employee e = (Employee) o;
        return id == e.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

✓ Class is immutable:

- final class
 - private final fields
 - No setters
-

Step 2 Use distinct()

```
import java.util.*;

public class ImmutableDistinctExample3 {

    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee(1, "A"),
            new Employee(1, "A"),
            new Employee(2, "B")
        );

        list.stream()
            .distinct()
            .forEach(e -> System.out.println(e.getId()));
    }
}
```

✓ Output:

1
2

✓ 5 What Happens If Object Is Mutable?

If you modify fields after inserting into stream:

```
// Bad practice
employee.setId(5);
```

⚠ This can break:

- Hash bucket consistency
- Duplicate detection
- Set behavior

That's why immutable objects are recommended.

✓ 6 Primitive Streams

Stream Type	Immutable?	equals()/hashCode()?
IntStream	Yes (primitive)	Not required
LongStream	Yes	Not required

DoubleStream	Yes	Not required

Example:

```
import java.util.stream.IntStream;

public class PrimitiveDistinctExample {
    public static void main(String[] args) {

        IntStream.of(1,2,2,3)
            .distinct()
            .forEach(System.out::println);
    }
}
```

✔ 7 Internal Concept (Simplified)

`distinct()` behaves like:

```
Set<T> seen = new HashSet<>();
if (seen.add(element)) {
    output(element);
}
```

If object is immutable:

- `hashCode()` never changes
 - `equals()` comparison remains stable
-

✔ 8 Complete Summary

Object Type	<code>distinct()</code> Behavior
String	Works perfectly
Integer	Works perfectly
LocalDate	Works perfectly
Custom Immutable Object	Must override <code>equals</code> & <code>hashCode</code>
Mutable Object	Risky / not recommended

❑ Interview-Ready Answer

The `distinct()` method takes no parameters and internally uses a `HashSet`. Therefore, it relies on stable `equals()` and `hashCode()` implementations. Immutable objects are ideal because their state cannot change after insertion, ensuring consistent duplicate detection. For custom immutable classes, overriding both methods is mandatory.

If you'd like next:

- ❑ Internal working of `distinct()`
- ⚙ Parallel stream behavior
- ❑ UML execution flow
- ❑ Java 1.5 equivalent
- ❑ `distinct()` vs `Collectors.toSet()` comparison

✔ Java Stream API `map()` — Transform Objects

❑ 1 Official Method Signature

`<R> Stream<R> map(Function<? super T, ? extends R> mapper)`

❑ `map()` accepts **only ONE input parameter**:

Stream Type	Accepted Parameter
Stream<T>	<code>Function<T, R></code>
IntStream	<code>IntUnaryOperator</code>
LongStream	<code>LongUnaryOperator</code>
DoubleStream	<code>DoubleUnaryOperator</code>

❑ 2 What is Function?

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

- ✓ Takes one input
 - ✓ Returns transformed output
-

✓ 3 All Input Forms Accepted by map()

1 Lambda Expression

```
import java.util.*;

public class MapExample1 {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("java", "stream");

        list.stream()
            .map(s -> s.toUpperCase())
            .forEach(System.out::println);
    }
}
```

- ✓ Input: `Function<String, String>`
-

2 Method Reference

```
list.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

- ✓ Cleaner syntax
-

3 Anonymous Class

```
list.stream()
    .map(new java.util.function.Function<String, String>() {
        @Override
        public String apply(String s) {
            return s.toUpperCase();
        }
    })
    .forEach(System.out::println);
```

4 Function Variable

```
java.util.function.Function<String, String> upper = s -> s.toUpperCase();
```

```
list.stream()  
    .map(upper)  
    .forEach(System.out::println);
```

✔ 4 Object Transformation Examples

Example 1 Convert String → Length (String → Integer)

```
list.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

✔ Function<String, Integer>

Example 2 Custom Object → Field Extraction

```
import java.util.*;  
  
class Employee {  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
}  
  
public class MapExample2 {  
    public static void main(String[] args) {  
  
        List<Employee> list = Arrays.asList(  
            new Employee("A", 30000),
```

```

        new Employee("B", 50000)
    );

    list.stream()
        .map(Employee::getName)
        .forEach(System.out::println);
}
}

```

✓ Function<Employee, String>

Example 3 Transform Object → New Object

```

list.stream()
    .map(e -> new Employee(e.name, e.salary + 1000))
    .forEach(e -> System.out.println(e.salary));

```

✓ Creates new transformed objects

✓ 5 Primitive Stream Variants

IntStream

```

import java.util.stream.IntStream;

public class MapPrimitiveExample {
    public static void main(String[] args) {

        IntStream.range(1, 5)
            .map(n -> n * 2)
            .forEach(System.out::println);
    }
}

```

// IntUnaryOperator

mapToInt / mapToLong / mapToDouble

```

list.stream()
    .mapToInt(String::length)
    .forEach(System.out::println);

```

Accepted Parameters:

Method	Functional Interface
mapToInt()	ToIntFunction<T>
mapToLong()	ToLongFunction<T>
mapToDouble()	ToDoubleFunction<T>

✔ 6 Function Default Methods

Function provides:

```
andThen()
compose()
identity()
```

Example:

```
Function<String, String> upper = String::toUpperCase;
Function<String, Integer> length = String::length;
```

```
list.stream()
    .map(upper.andThen(length))
    .forEach(System.out::println);
```

✔ 7 Complete List of Accepted Input Types

For Stream<T>.map()

- Function<T, R>
- Lambda returning R
- Method reference returning R
- Anonymous class
- Function variable

For Primitive Streams

- IntUnaryOperator
- LongUnaryOperator
- DoubleUnaryOperator

For mapToX()

- ToIntFunction<T>
- ToLongFunction<T>
- ToDoubleFunction<T>

❑ Interview-Ready Answer

The `map()` method in Java Stream API accepts a single parameter of type `Function<T, R>`. It transforms each element of the stream into another object. The input can be provided as a lambda expression, method reference, anonymous class, or Function variable. Primitive streams use specialized functional interfaces like `IntUnaryOperator`. The method returns a new Stream of transformed elements.

If you want next:

- ☐ Internal execution flow of `map()`
- ⚙ Bytecode-level explanation
- ☐ Parallel stream behavior
- ☐ UML execution diagram
- ☐ Java 1.5 equivalent implementation
- ☐ `map()` vs `flatMap()` deep comparison

✔ Java Stream API `map()` — Creating New Objects

❑ 1 Official Method Signature

`<R> Stream<R> map(Function<? super T, ? extends R> mapper)`

❑ `map()` accepts **only ONE parameter**:

Stream Type	Accepted Functional Interface
Stream<T>	<code>Function<T, R></code>
IntStream	<code>IntUnaryOperator</code>
LongStream	<code>LongUnaryOperator</code>
DoubleStream	<code>DoubleUnaryOperator</code>

✔ That means **any lambda / method reference that returns a NEW object** is valid.

✓ 2 What map() Does

- Takes each element
 - Applies transformation
 - Returns **new object**
 - Produces new Stream<R>
-

✓ 3 All Accepted Input Forms to Create New Objects

1 Lambda Expression Creating New Object

```
import java.util.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

public class MapNewObjectExample1 {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("A", "B");

        names.stream()
            .map(name -> new Employee(name, 50000))
            .forEach(e -> System.out.println(e.name));
    }
}
```

✓ Function<String, Employee>

2 Constructor Method Reference

```
names.stream()
    .map(Employee::new)
    .forEach(System.out::println);
```

⚠ Works only if constructor matches input type.

Example:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

names.stream()
    .map(Person::new)
    .forEach(p -> System.out.println(p.name));
```

3 Anonymous Class Creating New Object

```
names.stream()
    .map(new java.util.function.Function<String, Employee>() {
        @Override
        public Employee apply(String name) {
            return new Employee(name, 60000);
        }
    })
    .forEach(e -> System.out.println(e.name));
```

4 Function Variable

```
java.util.function.Function<String, Employee> creator =
    name -> new Employee(name, 70000);

names.stream()
    .map(creator)
    .forEach(e -> System.out.println(e.name));
```

✓ 4 Transform One Object to Another Object

Example: DTO Conversion

```
import java.util.*;

class User {
    int id;
    String name;
```

```

    User(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

class UserDTO {
    String name;

    UserDTO(String name) {
        this.name = name;
    }
}

public class MapNewObjectExample2 {
    public static void main(String[] args) {

        List<User> users = Arrays.asList(
            new User(1, "John"),
            new User(2, "Alice")
        );

        users.stream()
            .map(u -> new UserDTO(u.name))
            .forEach(dto -> System.out.println(dto.name));
    }
}

```

✓ Function<User, UserDTO>

✓ 5 Primitive Stream Object Creation

IntStream → Custom Object

```

import java.util.stream.IntStream;

class NumberWrapper {
    int value;

    NumberWrapper(int value) {
        this.value = value;
    }
}

```

```

public class MapPrimitiveToObject {
    public static void main(String[] args) {

        IntStream.range(1, 5)
            .mapToObj(NumberWrapper::new)
            .forEach(n -> System.out.println(n.value));
    }
}

```

- ✓ Uses mapToObj()
- ✓ Accepts IntFunction<R>

✓ 6 List of All Functional Interfaces Accepted

For Stream<T>.map()

- Function<T, R>
- Lambda returning new object
- Constructor reference
- Anonymous class
- Function variable

For Primitive Streams

Method	Functional Interface
map() (IntStream)	IntUnaryOperator
mapToObj()	IntFunction<R>
mapToInt()	ToIntFunction<T>
mapToLong()	ToLongFunction<T>
mapToDouble()	ToDoubleFunction<T>

✓ 7 Important Concept

- ✓ map() should be **pure transformation**
- ✓ Prefer creating new immutable objects
- ✓ Do NOT mutate existing objects inside map()

Bad practice:

```
.map(e -> { e.salary += 1000; return e; })
```

Good practice:

```
.map(e -> new Employee(e.name, e.salary + 1000))
```

❑ Interview-Ready Answer

The `map()` method accepts a single `Function<T, R>` parameter. It transforms each element into a new object and returns a new Stream. The function can be provided as a lambda, constructor reference, anonymous class, or Function variable. Primitive streams use specialized interfaces like `IntFunction`, `IntUnaryOperator`, and `ToIntFunction`. Best practice is to create new immutable objects instead of mutating existing ones.

If you'd like next:

- ❑ Internal working of `map()`
- ⚙ Bytecode-level explanation
- ❑ Parallel stream object creation behavior
- ❑ UML execution diagram
- ❑ `map()` vs `flatMap()` deep comparison
- ❑ Java 1.5 equivalent implementation

✔ Java Stream API `map()` — Calling Pure Functions

❑ 1 Official Method Signature

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

❑ `map()` accepts **only ONE input parameter**:

Stream Type	Accepted Functional Interface
Stream<T>	<code>Function<T, R></code>
IntStream	<code>IntUnaryOperator</code>
LongStream	<code>LongUnaryOperator</code>

DoubleStream	DoubleUnaryOperator
--------------	---------------------

✔ 2 What is a Pure Function?

A **pure function**:

- ✔ Depends only on input
- ✔ Returns same output for same input
- ✔ Has NO side effects
- ✔ Does NOT modify external state

Perfect for map().

✔ 3 All Accepted Input Forms to Call Pure Functions

1 Lambda Expression (Pure Function)

```
import java.util.*;

public class MapPureExample1 {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4);

        numbers.stream()
            .map(n -> n * n)
            .forEach(System.out::println);
    }
}
```

// pure function

✔ Function<Integer, Integer>

2 Static Method Reference (Pure)

```
import java.util.*;

public class MapPureExample2 {
```

```

public static int square(int n) {
    return n * n;
}

public static void main(String[] args) {

    List<Integer> numbers = Arrays.asList(1,2,3,4);

    numbers.stream()
        .map(MapPureExample2::square)
        .forEach(System.out::println);
}
}

```

- ✓ Method must return a value
 - ✓ Must not modify external state
-

3 Instance Method Reference (Pure)

```

import java.util.*;

public class MapPureExample3 {
    public static void main(String[] args) {

        List<String> words = Arrays.asList("java", "stream");

        words.stream()
            .map(String::toUpperCase)
            .forEach(System.out::println);
    }
}

```

- ✓ String::toUpperCase is pure (String is immutable)
-

4 Function Variable (Pure)

```

import java.util.*;
import java.util.function.Function;

public class MapPureExample4 {
    public static void main(String[] args) {

        Function<Integer, Integer> cube = n -> n * n * n;
    }
}

```

```
    Arrays.asList(1,2,3)
        .stream()
        .map(cube)
        .forEach(System.out::println);
    }
}
```

5 Anonymous Class (Pure)

```
numbers.stream()
    .map(new java.util.function.Function<Integer, Integer>() {
        @Override
        public Integer apply(Integer n) {
            return n * 2;
        }
    })
    .forEach(System.out::println);
```

✔ 4 Primitive Stream Pure Functions

IntStream

```
import java.util.stream.IntStream;

public class PrimitivePureExample {
    public static void main(String[] args) {

        IntStream.range(1,5)
            .map(n -> n + 10)
            .forEach(System.out::println);
    }
}
```

// IntUnaryOperator

mapToObj() with Pure Function

```
IntStream.range(1,4)
    .mapToObj(n -> "Number: " + n)
    .forEach(System.out::println);
```

✔ Uses IntFunction<R>

✓ 5 Function Default Methods (Pure Composition)

Function provides:

- `andThen()`
- `compose()`
- `identity()`

Example:

```
Function<Integer, Integer> square = n -> n * n;  
Function<Integer, Integer> add10 = n -> n + 10;
```

```
Arrays.asList(1,2,3)  
    .stream()  
    .map(square.andThen(add10))  
    .forEach(System.out::println);
```

✓ Pure function chaining

✓ 6 Complete List of Accepted Input Types

For `Stream<T>.map()`

- `Function<T, R>`
 - Lambda returning `R`
 - Static method reference
 - Instance method reference
 - Constructor reference
 - Anonymous class
 - Function variable
-

For Primitive Streams

Method	Functional Interface
<code>IntStream.map()</code>	<code>IntUnaryOperator</code>
<code>LongStream.map()</code>	<code>LongUnaryOperator</code>
<code>DoubleStream.map()</code>	<code>DoubleUnaryOperator</code>
<code>mapToObj()</code>	<code>IntFunction<R></code>

mapToInt()	ToIntFunction<T>
mapToLong()	ToLongFunction<T>
mapToDouble()	ToDoubleFunction<T>

✔ 7 What NOT to Do (Impure Function)

✗ Side effects inside map():

```
.map(n -> {
    System.out.println(n);           // side effect
    return n * 2;
})
```

✗ Mutating external object:

```
.map(e -> {
    e.salary += 1000;               // modifies state
    return e;
})
```

✔ Prefer:

```
.map(e -> new Employee(e.name, e.salary + 1000))
```

□ Interview-Ready Answer

The map() method accepts a single parameter of type Function<T, R>. Any pure function—provided as a lambda, method reference, constructor reference, anonymous class, or Function variable—can be passed to map(). The function must return a value and should ideally be side-effect free.

Primitive streams use specialized functional interfaces like IntUnaryOperator. Pure functions are recommended to maintain functional programming principles and safe parallel execution.

If you'd like next:

- ☐ Internal execution flow of map()
- ☒ Bytecode-level explanation
- ☐ Parallel stream behavior with pure functions
- ☐ UML execution diagram
- ☐ map() vs flatMap() deep comparison
- ☐ Java 1.5 equivalent implementation

✔ Java Stream API flatMap() — Returning a Stream

❑ 1 Official Method Signature

<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)

❑ flatMap() accepts **only ONE parameter**:

Stream Type	Accepted Functional Interface
Stream<T>	Function<T, Stream<R>>
IntStream	IntFunction<IntStream>
LongStream	LongFunction<LongStream>
DoubleStream	DoubleFunction<DoubleStream>

✔ The function **must return a Stream**

✔ 2 What flatMap() Does

- Takes each element
 - Converts it into a **Stream**
 - Flattens all streams into **one single stream**
-

✔ 3 All Input Forms Accepted by flatMap()

1 Lambda Returning Stream

```
import java.util.*;
```

```
public class FlatMapExample1 {  
    public static void main(String[] args) {  
  
        List<List<String>> list = Arrays.asList(  
            Arrays.asList("A", "B"),
```

```

        Arrays.asList("C", "D")
    );

    list.stream()
        .flatMap(innerList -> innerList.stream())
        .forEach(System.out::println);
}
}

```

✓ `Function<List<String>, Stream<String>>`

2 Method Reference Returning Stream

```

list.stream()
    .flatMap(List::stream)
    .forEach(System.out::println);

```

✓ Cleaner and preferred

3 Static Method Returning Stream

```

import java.util.stream.Stream;

public class FlatMapExample2 {

    public static Stream<String> splitWords(String line) {
        return Arrays.stream(line.split(" "));
    }

    public static void main(String[] args) {

        List<String> lines = Arrays.asList("Java Stream", "Flat Map");

        lines.stream()
            .flatMap(FlatMapExample2::splitWords)
            .forEach(System.out::println);
    }
}

```

✓ Method must return `Stream<R>`

4 Anonymous Class

```
list.stream()
    .flatMap(new java.util.function.Function<List<String>, Stream<String>>() {
        @Override
        public Stream<String> apply(List<String> l) {
            return l.stream();
        }
    })
    .forEach(System.out::println);
```

5 Function Variable

```
import java.util.function.Function;

Function<List<String>, Stream<String>> mapper = l -> l.stream();

list.stream()
    .flatMap(mapper)
    .forEach(System.out::println);
```

✔ 4 Custom Object Example

```
import java.util.*;

class Order {
    List<String> items;

    Order(List<String> items) {
        this.items = items;
    }

    public List<String> getItems() {
        return items;
    }
}

public class FlatMapExample3 {

    public static void main(String[] args) {

        List<Order> orders = Arrays.asList(
            new Order(Arrays.asList("Pen", "Book")),
            new Order(Arrays.asList("Laptop"))
        );
```

```

        orders.stream()
            .flatMap(order -> order.getItems().stream())
            .forEach(System.out::println);
    }
}

```

✓ Function<Order, Stream<String>>

✓ 5 Primitive Stream flatMap Variants

IntStream flatMap

```

import java.util.stream.IntStream;

public class FlatMapPrimitiveExample {
    public static void main(String[] args) {

        IntStream.of(1, 2, 3)
            .flatMap(n -> IntStream.range(0, n))
            .forEach(System.out::println);
    }
}

```

✓ Uses IntFunction<IntStream>

mapToObj + flatMap

```

IntStream.range(1,4)
    .mapToObj(n -> Arrays.asList(n, n*10))
    .flatMap(List::stream)
    .forEach(System.out::println);

```

✓ 6 Complete List of Accepted Input Types

For Stream<T>.flatMap()

- Function<T, Stream<R>>
- Lambda returning Stream
- Method reference returning Stream
- Static method returning Stream
- Anonymous class

- Function variable

For Primitive Streams

Method	Functional Interface
IntStream.flatMap()	IntFunction<IntStream>
LongStream.flatMap()	LongFunction<LongStream>
DoubleStream.flatMap()	DoubleFunction<DoubleStream>

✔ 7 map() vs flatMap()

map()	flatMap()
Returns single object	Returns Stream
Produces nested stream	Flattens nested stream
Function<T, R>	Function<T, Stream>

Example difference:

```
.map(List::stream)           // Stream<Stream<String>>
.flatMap(List::stream)       // Stream<String>
```

❑ Interview-Ready Answer

The flatMap() method accepts a single parameter of type Function<T, Stream<R>>. The function must return a Stream, which is then flattened into a single output stream. It can be provided as a lambda, method reference, static method, anonymous class, or Function variable. Primitive streams use specialized interfaces like IntFunction<IntStream>. Unlike map(), flatMap() removes nested stream structure.

If you'd like next:

- ☐ Internal execution flow of flatMap()
- ⚙ Bytecode-level explanation
- ☐ Parallel stream behavior

- ☐ UML execution diagram
- ☐ flatMap() vs map() deep internal comparison
- ☐ Java 1.5 equivalent implementation

✔ Java Stream API flatMap() — Split Data (One → Many)

☐ 1 Official Method Signature

<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)

☐ flatMap() accepts **one parameter only**:

Stream Type	Accepted Functional Interface
Stream<T>	Function<T, Stream<R>>
IntStream	IntFunction<IntStream>
LongStream	LongFunction<LongStream>
DoubleStream	DoubleFunction<DoubleStream>

✔ The function **must return a Stream**

✔ Used when one element needs to be split into multiple elements

✔ 2 What "Split Data" Means

Example:

"Java Stream API"

Split into:

Java
Stream
API

So:

- One input

- Multiple outputs
 - Return as Stream
 - Flatten all results
-

✔ 3 All Input Forms Accepted by flatMap() (For Splitting)

1 Lambda Returning Stream (Most Common)

Example: Split sentences into words

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitExample1 {
    public static void main(String[] args) {

        List<String> lines = Arrays.asList(
            "Java Stream API",
            "FlatMap Example"
        );

        lines.stream()
            .flatMap(line -> Arrays.stream(line.split(" ")))
            .forEach(System.out::println);
    }
}
```

✔ Function<String, Stream<String>>

2 Method Reference Returning Stream

```
lines.stream()
    .flatMap(line -> Arrays.stream(line.split(" ")))
    .forEach(System.out::println);
```

Or using helper method:

```
public static Stream<String> splitWords(String line) {
    return Arrays.stream(line.split(" "));
}
```

```
lines.stream()
```

```
.flatMap(FlatMapSplitExample1::splitWords)
.forEach(System.out::println);
```

3 Using List::stream (Nested List Splitting)

```
import java.util.*;

public class FlatMapSplitExample2 {
    public static void main(String[] args) {

        List<List<Integer>> data = Arrays.asList(
            Arrays.asList(1,2),
            Arrays.asList(3,4)
        );

        data.stream()
            .flatMap(List::stream)
            .forEach(System.out::println);
    }
}
```

✓ Splits nested list

4 Anonymous Class

```
data.stream()
    .flatMap(new java.util.function.Function<List<Integer>, Stream<Integer>>() {
        @Override
        public Stream<Integer> apply(List<Integer> list) {
            return list.stream();
        }
    })
    .forEach(System.out::println);
```

5 Function Variable

```
import java.util.function.Function;

Function<String, Stream<String>> splitter =
    line -> Arrays.stream(line.split(" "));

lines.stream()
    .flatMap(splitter)
    .forEach(System.out::println);
```

✔ 4 Splitting Characters

```
List<String> words = Arrays.asList("Java", "API");

words.stream()
    .flatMap(word -> word.chars()
        .mapToObj(c -> String.valueOf((char) c)))
    .forEach(System.out::println);
```

✔ One word → multiple characters

✔ 5 Primitive Stream flatMap Variants

IntStream flatMap

```
import java.util.stream.IntStream;

public class FlatMapPrimitiveSplit {
    public static void main(String[] args) {

        IntStream.of(1, 2, 3)
            .flatMap(n -> IntStream.range(0, n))
            .forEach(System.out::println);
    }
}
```

✔ Uses IntFunction<IntStream>

✔ 6 Complete List of Accepted Input Types

For Stream<T>.flatMap()

- Lambda returning Stream<R>
 - Method reference returning Stream<R>
 - Static method returning Stream<R>
 - Instance method returning Stream<R>
 - Anonymous class
 - Function variable
-

For Primitive Streams

Method	Functional Interface
IntStream.flatMap()	IntFunction<IntStream>
LongStream.flatMap()	LongFunction<LongStream>
DoubleStream.flatMap()	DoubleFunction<DoubleStream>

✔ 7 Important Difference

map()	flatMap()
One → One	One → Many
Function<T, R>	Function<T, Stream>
Produces nested structure	Flattens nested structure

Example:

```
.map(line -> Arrays.stream(line.split(" ")))
// Stream<Stream<String>>
```

```
.flatMap(line -> Arrays.stream(line.split(" ")))
// Stream<String>
```

□ Interview-Ready Answer

The flatMap() method accepts a single parameter of type Function<T, Stream<R>>. It is used when one element must be split into multiple elements. The function must return a Stream, and the resulting nested streams are flattened into one stream. It can be provided as a lambda, method reference, static method, anonymous class, or Function variable. Primitive streams use specialized interfaces like IntFunction<IntStream>.

If you'd like next:

- ☐ Internal execution flow of flatMap()
- ☒ Bytecode-level explanation
- ☐ Parallel stream behavior
- ☐ UML execution diagram
- ☐ flatMap() vs map() internal comparison
- ☐ Java 1.5 equivalent implementation

✔ Java Stream API flatMap() — Expand Collections

❑ 1 Official Method Signature

`<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`

❑ **flatMap()** accepts only ONE parameter:

Stream Type	Accepted Functional Interface
Stream<T>	Function<T, Stream<R>>
IntStream	IntFunction<IntStream>
LongStream	LongFunction<LongStream>
DoubleStream	DoubleFunction<DoubleStream>

✔ The function **must return a Stream**

✔ Used to expand nested collections into a single flattened stream

✔ 2 What “Expand Collections” Means

Example structure:

`List<List<String>>`

Using `flatMap()`:

`Stream<String>`

So:

- One outer element
 - Expands into inner collection
 - Flattens all inner collections
-

✔ 3 All Input Forms Accepted by flatMap()

1 Lambda Returning Collection Stream (Most Common)

Expand List<List<String>>

```
import java.util.*;

public class FlatMapExpandExample1 {
    public static void main(String[] args) {

        List<List<String>> data = Arrays.asList(
            Arrays.asList("A", "B"),
            Arrays.asList("C", "D")
        );

        data.stream()
            .flatMap(list -> list.stream())
            .forEach(System.out::println);
    }
}
```

✓ Function<List<String>, Stream<String>>

2 Method Reference (Collection::stream)

```
data.stream()
    .flatMap(List::stream)
    .forEach(System.out::println);
```

✓ Cleanest and preferred approach

3 Static Helper Method Returning Stream

```
import java.util.stream.Stream;

public class FlatMapExpandExample2 {

    public static Stream<String> expand(List<String> list) {
        return list.stream();
    }

    public static void main(String[] args) {
```

```

List<List<String>> data = Arrays.asList(
    Arrays.asList("X", "Y"),
    Arrays.asList("Z")
);

data.stream()
    .flatMap(FlatMapExpandExample2::expand)
    .forEach(System.out::println);
}
}

```

✓ Method must return Stream<R>

4 Anonymous Class

```

data.stream()
    .flatMap(new java.util.function.Function<List<String>, Stream<String>>() {
        @Override
        public Stream<String> apply(List<String> list) {
            return list.stream();
        }
    })
    .forEach(System.out::println);

```

5 Function Variable

```

import java.util.function.Function;

Function<List<String>, Stream<String>> expander = List::stream;

data.stream()
    .flatMap(expander)
    .forEach(System.out::println);

```

✓ 4 Expand Collection from Custom Object

```

import java.util.*;

class Order {
    private List<String> items;

    Order(List<String> items) {
        this.items = items;
    }
}

```

```

    public List<String> getItems() {
        return items;
    }
}

public class FlatMapExpandExample3 {

    public static void main(String[] args) {

        List<Order> orders = Arrays.asList(
            new Order(Arrays.asList("Pen", "Book")),
            new Order(Arrays.asList("Laptop"))
        );

        orders.stream()
            .flatMap(order -> order.getItems().stream())
            .forEach(System.out::println);
    }
}

```

✓ Function<Order, Stream<String>>

✓ 5 Expand Arrays

```

import java.util.*;

public class FlatMapExpandExample4 {
    public static void main(String[] args) {

        List<String[]> arrays = Arrays.asList(
            new String[]{"A", "B"},
            new String[]{"C"}
        );

        arrays.stream()
            .flatMap(arr -> Arrays.stream(arr))
            .forEach(System.out::println);
    }
}

```

✓ 6 Primitive Stream Expansion

IntStream Example

```
import java.util.stream.IntStream;

public class FlatMapPrimitiveExpand {
    public static void main(String[] args) {

        IntStream.of(2,3)
            .flatMap(n -> IntStream.range(1, n + 1))
            .forEach(System.out::println);
    }
}
```

✓ Uses IntFunction<IntStream>

✓ 7 Complete List of Accepted Input Types

For Stream<T>.flatMap()

- Lambda returning Stream<R>
- Method reference returning Stream<R>
- Static method returning Stream<R>
- Instance method returning Stream<R>
- Anonymous class
- Function variable

For Primitive Streams

Method	Functional Interface
IntStream.flatMap()	IntFunction<IntStream>
LongStream.flatMap()	LongFunction<LongStream>
DoubleStream.flatMap()	DoubleFunction<DoubleStream>

✓ 8 map() vs flatMap() (Collection Case)

```
.map(List::stream)
// Result: Stream<Stream<String>>
```

```
.flatMap(List::stream)
// Result: Stream<String>
```

✓ flatMap() removes nested structure

❑ Interview-Ready Answer

The flatMap() method accepts a single parameter of type Function<T, Stream<R>>. It is used to expand nested collections into a single flattened stream. The function must return a Stream and can be provided as a lambda, method reference, static method, anonymous class, or Function variable. Primitive streams use specialized interfaces like IntFunction<IntStream>. Unlike map(), flatMap() removes nested stream layers.

If you'd like next:

- ☐ Internal working of flatMap()
- ⚙ Bytecode-level explanation
- ☐ Parallel stream behavior
- ☐ UML execution diagram
- ☐ flatMap() vs map() deep internal comparison
- ☐ Java 1.5 equivalent implementation

✓✓ Java Stream API sorted() — Natural Ordering

❑ 1 Official Method Signatures

```
Stream<T> sorted()  
Stream<T> sorted(Comparator<? super T> comparator)
```

For **natural ordering**, we use:

```
sorted()
```

❑ No input parameters required

✓✓ 2 What is Natural Ordering?

Natural ordering means:

- The class implements Comparable<T>
- Uses compareTo() method internally

Examples of classes that already support natural ordering:

- String
 - Integer
 - Double
 - Long
 - LocalDate
 - etc.
-

✔ 3 How sorted() Works Internally

When using:

`stream.sorted()`

Java:

- Checks if elements implement Comparable
 - Calls `compareTo()` method
 - Sorts elements accordingly
-

✔ 4 Example 1 — Integer Natural Ordering

```
import java.util.*;
```

```
public class SortedExample1 {  
    public static void main(String[] args) {  
  
        List<Integer> numbers = Arrays.asList(30, 10, 20);  
  
        numbers.stream()  
            .sorted()  
            .forEach(System.out::println);  
    }  
}
```

✔ Output:

```
10  
20  
30
```

✔ Uses `Integer.compareTo()`

✔ 5 Example 2 — String Natural Ordering

```
import java.util.*;

public class SortedExample2 {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("Banana", "Apple", "Cherry");

        names.stream()
            .sorted()
            .forEach(System.out::println);
    }
}
```

✔ Output:

Apple
Banana
Cherry

✔ Uses lexicographical ordering

✔ 6 Example 3 — LocalDate Natural Ordering

```
import java.time.LocalDate;
import java.util.*;

public class SortedExample3 {
    public static void main(String[] args) {

        List<LocalDate> dates = Arrays.asList(
            LocalDate.of(2025, 1, 1),
            LocalDate.of(2023, 1, 1)
        );

        dates.stream()
            .sorted()
            .forEach(System.out::println);
    }
}
```

✔ Uses LocalDate.compareTo()

✔ 7 Custom Object with Natural Ordering

To use `sorted()` without comparator:

✔ Must implement `Comparable<T>`

Step 1 Implement Comparable

```
import java.util.*;

class Employee implements Comparable<Employee> {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }
}
```

Step 2 Use sorted()

```
public class SortedExample4 {
    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee(3, "C"),
            new Employee(1, "A"),
            new Employee(2, "B")
        );

        list.stream()
            .sorted()
            .forEach(e -> System.out.println(e.id));
    }
}
```

✓ Output:

1
2
3

✓ 8 What Happens If Comparable Not Implemented?

`list.stream().sorted();`

✗ Runtime error:

`ClassCastException`

Because Java cannot compare objects.

✓ 9 Primitive Streams Natural Ordering

Primitive streams always use natural ordering:

`import java.util.stream.IntStream;`

```
public class PrimitiveSortedExample {  
    public static void main(String[] args) {  
  
        IntStream.of(3,1,2)  
            .sorted()  
            .forEach(System.out::println);  
    }  
}
```

✓ Works automatically

✓ 10 Complete List of Input Parameters for Natural Ordering

For **natural ordering** only:

Method	Parameter
<code>sorted()</code>	✗ No parameter
<code>IntStream.sorted()</code>	✗ No parameter

LongStream.sorted()	✗ No parameter
DoubleStream.sorted()	✗ No parameter

✓ Relies on:

- Comparable.compareTo()

❑ Interview-Ready Answer

The sorted() method with natural ordering accepts no parameters. It sorts elements based on their natural order, which requires the elements to implement Comparable. Internally, it calls the compareTo() method. If the elements do not implement Comparable, a ClassCastException is thrown at runtime. Primitive streams use built-in natural ordering automatically.

If you'd like next:

- ☐ Internal working of sorted()
- ⚙ Bytecode-level explanation
- ☐ Parallel stream sorting behavior
- ☐ UML execution diagram
- ☐ sorted() with Comparator deep explanation
- ☐ Java 1.5 equivalent sorting

✓ Java Stream API sorted() — Custom Comparator

❑ 1 Official Method Signature

Stream<T> sorted(Comparator<? super T> comparator)

❑ When using **custom sorting**, sorted() accepts **only ONE parameter**:

Stream Type	Accepted Functional Interface
Stream<T>	Comparator<? super T>

⚠ Primitive streams (IntStream, LongStream, DoubleStream) **do NOT accept custom comparators** — they only support natural ordering.

✔ 2 What is a Comparator?

```
@FunctionalInterface
public interface Comparator<T> {
    int compare(T o1, T o2);
}
```

✔ Takes two inputs

✔ Returns:

- negative $\rightarrow o1 < o2$
 - zero $\rightarrow$ equal
 - positive $\rightarrow o1 > o2$
-

✔ 3 All Input Forms Accepted by sorted(Comparator)

1 Lambda Comparator (Most Common)

```
import java.util.*;

public class SortedComparatorExample1 {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("Banana", "Apple", "Cherry");

        names.stream()
            .sorted((a, b) -> a.length() - b.length())
            .forEach(System.out::println);
    }
}
```

✔ Sort by length

2 Method Reference Comparator

Using built-in comparator methods:

```
names.stream()
    .sorted(Comparator.comparing(String::length))
    .forEach(System.out::println);
```

✓ Cleaner and preferred

3 Anonymous Class

```
names.stream()
    .sorted(new Comparator<String>() {
        @Override
        public int compare(String a, String b) {
            return a.length() - b.length();
        }
    })
    .forEach(System.out::println);
```

4 Comparator Variable

```
Comparator<String> byLength = (a, b) -> a.length() - b.length();
```

```
names.stream()
    .sorted(byLength)
    .forEach(System.out::println);
```

5 Static Comparator Method

```
public static int compareByLength(String a, String b) {
    return a.length() - b.length();
}
```

```
names.stream()
    .sorted(SortedComparatorExample1::compareByLength)
    .forEach(System.out::println);
```

✓ 4 Custom Object Example

Sort Employees by Salary

```
import java.util.*;
```

```

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }
}

public class SortedComparatorExample2 {
    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee("A", 50000),
            new Employee("B", 30000),
            new Employee("C", 70000)
        );

        list.stream()
            .sorted(Comparator.comparing(Employee::getSalary))
            .forEach(e -> System.out.println(e.salary));
    }
}

```

✔ 5 Reverse Order Sorting

```

list.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .forEach(e -> System.out.println(e.salary));

```

✔ 6 Multiple Field Sorting (Chaining)

```

list.stream()
    .sorted(Comparator.comparing(Employee::getSalary)
        .thenComparing(e -> e.name))
    .forEach(e -> System.out.println(e.name));

```

✔ 7 Built-in Comparator Methods

Common helper methods:

- `Comparator.comparing()`
 - `thenComparing()`
 - `reversed()`
 - `naturalOrder()`
 - `reverseOrder()`
 - `nullsFirst()`
 - `nullsLast()`
-

Example with null handling

```
names.stream()  
    .sorted(Comparator.nullsLast(String::compareTo))  
    .forEach(System.out::println);
```

✔ 8 Primitive Stream Limitation

✗ Not allowed:

```
IntStream.of(3,1,2)  
    .sorted((a,b) -> b - a);
```

// Compilation error

✔ Only natural ordering supported in primitive streams.

✔ 9 Complete List of Accepted Input Types

For `Stream<T>.sorted()` with custom sorting:

- Lambda `(a,b) -> int`
 - Method reference returning int
 - Anonymous class
 - Comparator variable
 - `Comparator.comparing()`
 - `Comparator.comparingInt()`
 - `Comparator.comparingLong()`
 - `Comparator.comparingDouble()`
-

□ Interview-Ready Answer

The `sorted(Comparator)` method accepts a single parameter of type `Comparator<T>`. The comparator defines custom ordering logic by implementing the `compare()` method. It can be provided as a lambda expression, method reference, anonymous class, Comparator variable, or

using built-in helper methods like `Comparator.comparing()`. Primitive streams do not support custom comparators.

If you'd like next:

- ☐ Internal working of `sorted()`
- ☒ Bytecode-level explanation
- ☐ Parallel stream sorting behavior
- ☐ UML execution diagram
- ☐ Natural ordering vs `Comparator` deep comparison
- ☐ Java 1.5 equivalent sorting

✔ Java Stream API `sorted()` — Multi-Field Comparison

☐ 1 Official Method Signature

`Stream<T> sorted(Comparator<? super T> comparator)`

☐ For multi-field comparison, `sorted()` accepts **only ONE parameter**:

Method	Accepted Type
<code>sorted(...)</code>	<code>Comparator<? super T></code>

✔ Multi-field sorting is achieved using:

- `Comparator.comparing(...)`
 - `thenComparing(...)`
 - `thenComparingInt(...)`
 - `thenComparingLong(...)`
 - `thenComparingDouble(...)`
 - `reversed()`
 - `nullsFirst()`
 - `nullsLast()`
-

✔ 2 Why Multi-Field Sorting?

Example:

Sort employees by:

1. Salary (ascending)
2. Name (ascending)
3. Age (descending)

✓ 3 Example — Custom Object

Step 1 Create Class

```
import java.util.*;

class Employee {
    String name;
    double salary;
    int age;

    Employee(String name, double salary, int age) {
        this.name = name;
        this.salary = salary;
        this.age = age;
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }
    public int getAge() { return age; }
}
```

Step 2 Multi-Field Sorting

```
import java.util.*;

public class SortedMultiFieldExample {

    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
            new Employee("John", 50000, 30),
            new Employee("Alice", 50000, 25),
            new Employee("Bob", 30000, 40)
        );

        list.stream()
            .sorted(
```

```

        Comparator.comparing(Employee::getSalary)
                    .thenComparing(Employee::getName)
                    .thenComparing(Employee::getAge)
    )
    .forEach(e ->
        System.out.println(e.name + " " + e.salary + " " + e.age)
    );
}
}

```

✓ Sort order:

1. Salary ASC
2. Name ASC
3. Age ASC

✓ 4 Multi-Field with Mixed Order (ASC + DESC)

Example:

- Salary ASC
- Age DESC

```

list.stream()
    .sorted(
        Comparator.comparing(Employee::getSalary)
                    .thenComparing(
                        Comparator.comparing(Employee::getAge).reversed()
                    )
    )
    .forEach(e -> System.out.println(e.name));

```

✓ 5 Using Primitive-Specific Methods (Performance Optimized)

```

list.stream()
    .sorted(
        Comparator.comparingDouble(Employee::getSalary)
                    .thenComparingInt(Employee::getAge)
    )
    .forEach(e -> System.out.println(e.name));

```

✓ Avoids boxing/unboxing

✓ 6 Lambda-Based Multi-Field Comparator

```
list.stream()
    .sorted((e1, e2) -> {
        int salaryCompare = Double.compare(e1.salary, e2.salary);
        if (salaryCompare != 0) return salaryCompare;

        int nameCompare = e1.name.compareTo(e2.name);
        if (nameCompare != 0) return nameCompare;

        return Integer.compare(e1.age, e2.age);
    })
    .forEach(e -> System.out.println(e.name));
```

✓ Manual multi-level comparison

✓ 7 Handling Null Fields

```
list.stream()
    .sorted(
        Comparator.comparing(
            Employee::getName,
            Comparator.nullsLast(String::compareTo)
        )
    )
    .forEach(e -> System.out.println(e.name));
```

✓ 8 Complete List of Accepted Input Forms

For sorted() multi-field comparison:

- Lambda (a,b) -> int
- Method reference returning int
- Anonymous class
- Comparator variable
- Comparator.comparing()
- Comparator.comparingInt()
- Comparator.comparingLong()
- Comparator.comparingDouble()
- thenComparing()
- thenComparingInt()
- thenComparingLong()
- thenComparingDouble()
- reversed()
- nullsFirst()
- nullsLast()

✔ 9 Primitive Streams Limitation

✗ Not supported:

```
IntStream.of(1,2,3).sorted((a,b) -> b-a);           // Compile error
```

✔ Primitive streams support only natural ordering.

❑ Interview-Ready Answer

The `sorted(Comparator)` method accepts a single parameter of type `Comparator<T>`. Multi-field sorting is implemented using `Comparator.comparing()` combined with `thenComparing()` methods. Primitive-specific methods like `comparingInt()` improve performance. The comparator can be written as a lambda, method reference, anonymous class, or built using the `Comparator` helper methods. Primitive streams do not support custom multi-field comparators.

If you'd like next:

- ☐ Internal working of `sorted()`
- ⚙ Bytecode-level explanation
- ☐ Parallel stream sorting behavior
- ☐ UML execution diagram
- ☐ Performance comparison (`Comparator` vs `Comparable`)
- ☐ Java 1.5 equivalent multi-field sorting

✔ Java Stream API `peek()` — Logging / Debugging

❑ 1 Official Method Signature

`Stream<T> peek(Consumer<? super T> action)`

❑ `peek()` accepts **only ONE** parameter:

Stream Type	Accepted Functional Interface
Stream<T>	<code>Consumer<T></code>
IntStream	<code>IntConsumer</code>

LongStream	LongConsumer
DoubleStream	DoubleConsumer

- ✓ The function must return **void**
 - ✓ Used mainly for **logging / debugging**
-

✓ 2 What is Consumer?

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

- ✓ Takes one input
 - ✓ Returns nothing (void)
-

✓ 3 All Input Forms Accepted by peek()

1 Lambda Expression (Most Common)

```
import java.util.*;

public class PeekExample1 {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Stream", "API");

        list.stream()
            .peek(s -> System.out.println("Processing: " + s))
            .forEach(System.out::println);
    }
}
```

- ✓ Consumer<String>
-

2 Method Reference (System.out::println)

```
list.stream()  
    .peek(System.out::println)  
    .forEach(s -> {});
```

✓ Clean logging

3 Static Logging Method

```
public static void log(String s) {  
    System.out.println("LOG: " + s);  
}
```

```
list.stream()  
    .peek(PeekExample1::log)  
    .forEach(s -> {});
```

✓ Method must return void

4 Anonymous Class

```
list.stream()  
    .peek(new java.util.function.Consumer<String>() {  
        @Override  
        public void accept(String s) {  
            System.out.println("Debug: " + s);  
        }  
    })  
    .forEach(s -> {});
```

5 Consumer Variable

```
import java.util.function.Consumer;  
  
Consumer<String> logger = s -> System.out.println("Value: " + s);  
  
list.stream()  
    .peek(logger)  
    .forEach(s -> {});
```

✓ 4 Using peek() Between Operations (Pipeline Logging)

```
import java.util.*;

public class PeekPipelineExample {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4);

        numbers.stream()
            .peek(n -> System.out.println("Original: " + n))
            .map(n -> n * 2)
            .peek(n -> System.out.println("After Map: " + n))
            .filter(n -> n > 4)
            .peek(n -> System.out.println("After Filter: " + n))
            .forEach(System.out::println);
    }
}
```

✓ Shows pipeline execution

✓ 5 Primitive Stream peek()

IntStream

```
import java.util.stream.IntStream;

public class PeekPrimitiveExample {
    public static void main(String[] args) {

        IntStream.range(1,5)
            .peek(n -> System.out.println("Number: " + n))
            .map(n -> n * 2)
            .forEach(System.out::println);
    }
}
```

✓ Uses IntConsumer

✓ 6 Using Real Logging Framework

Example with java.util.logging.Logger:

```
import java.util.logging.Logger;
```

```
Logger logger = Logger.getLogger("MyLogger");
```

```
list.stream()  
    .peek(logger::info)  
    .forEach(s -> {});
```

✓ `Logger::info` matches `Consumer<String>`

✓ 7 Important Notes

✓ `peek()` is:

- Intermediate operation
- Lazy (executes only if terminal operation exists)
- Intended mainly for debugging

✗ Not recommended for:

- Business logic
 - State mutation
-

✗ Bad Practice (Side Effects)

```
list.stream()  
    .peek(s -> list.add("New"))  
    .forEach(System.out::println);
```

// Dangerous

May cause:

- `ConcurrentModificationException`
 - Unexpected behavior
-

✓ 8 Complete List of Accepted Input Types

For `peek()`:

- Lambda returning void
- Method reference returning void
- Static method reference
- Instance method reference

- Anonymous class
- Consumer variable

Primitive variants:

Method	Functional Interface
IntStream.peek()	IntConsumer
LongStream.peek()	LongConsumer
DoubleStream.peek()	DoubleConsumer

☐ Interview-Ready Answer

The peek() method accepts a single parameter of type Consumer<T>. It is mainly used for debugging and logging within a stream pipeline. The consumer must return void and can be provided as a lambda, method reference, anonymous class, or Consumer variable. Primitive streams use specialized interfaces like IntConsumer. Since peek() is lazy, it executes only when a terminal operation is invoked.

If you'd like next:

- ☐ Internal execution flow of peek()
- ☒ Bytecode-level explanation
- ☐ Parallel stream behavior with peek()
- ☐ UML execution diagram
- ☐ peek() vs forEach() deep comparison
- ☐ Java 1.5 equivalent logging approach

✔ Java Stream API peek() — Debugging

☐ 1 Official Method Signature

Stream<T> peek(Consumer<? super T> action)

☐ peek() accepts **only ONE input parameter**:

Stream Type	Accepted Functional Interface
-------------	-------------------------------

Stream<T>	Consumer<T>
IntStream	IntConsumer
LongStream	LongConsumer
DoubleStream	DoubleConsumer

- ✓ The function must return **void**
- ✓ Used mainly for **debugging stream pipelines**

✓ 2 What is Consumer?

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

- ✓ Takes one argument
- ✓ Returns nothing (void)

✓ 3 All Input Forms Accepted by peek() (For Debugging)

1 Lambda Expression (Most Common)

```
import java.util.*;

public class PeekDebugExample1 {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4);

        numbers.stream()
            .peek(n -> System.out.println("Before map: " + n))
            .map(n -> n * 2)
            .peek(n -> System.out.println("After map: " + n))
            .forEach(System.out::println);
    }
}
```

✓ Consumer<Integer>

2 Method Reference (System.out::println)

```
numbers.stream()
    .peek(System.out::println)
    .map(n -> n * 2)
    .forEach(System.out::println);
```

✓ Clean and simple debugging

3 Static Debug Method

```
public static void debug(Integer n) {
    System.out.println("Debug value: " + n);
}
```

```
numbers.stream()
    .peek(PeekDebugExample1::debug)
    .forEach(System.out::println);
```

✓ Method must return void

4 Anonymous Class

```
numbers.stream()
    .peek(new java.util.function.Consumer<Integer>() {
        @Override
        public void accept(Integer n) {
            System.out.println("Value: " + n);
        }
    })
    .forEach(System.out::println);
```

5 Consumer Variable

```
import java.util.function.Consumer;
```

```
Consumer<Integer> logger = n -> System.out.println("Processing: " + n);
```

```
numbers.stream()
    .peek(logger)
    .forEach(System.out::println);
```

✔ 4 Debugging Multiple Stages in Pipeline

```
import java.util.*;

public class PeekDebugExample2 {
    public static void main(String[] args) {

        List<String> words = Arrays.asList("java", "stream", "api");

        words.stream()
            .peek(w -> System.out.println("Original: " + w))
            .filter(w -> w.length() > 3)
            .peek(w -> System.out.println("After filter: " + w))
            .map(String::toUpperCase)
            .peek(w -> System.out.println("After map: " + w))
            .forEach(System.out::println);
    }
}
```

✔ Useful to understand execution order

✔ 5 Primitive Stream Debugging

IntStream Example

```
import java.util.stream.IntStream;

public class PeekPrimitiveDebug {
    public static void main(String[] args) {

        IntStream.range(1,5)
            .peek(n -> System.out.println("Before: " + n))
            .map(n -> n * 10)
            .peek(n -> System.out.println("After: " + n))
            .forEach(System.out::println);
    }
}
```

✔ Uses IntConsumer

✔ 6 Using Logger Framework

```
import java.util.logging.Logger;

Logger logger = Logger.getLogger("MyLogger");

numbers.stream()
    .peek(logger::info)
    .forEach(n -> {});
```

✔ `Logger::info` matches `Consumer<String>`

✔ 7 Important Debugging Concepts

✔ `peek()` is:

- Intermediate operation
- Lazy (runs only when terminal operation is called)
- Mainly for debugging

✗ Not meant for modifying data

✗ Wrong Usage (Side Effects)

```
numbers.stream()
    .peek(n -> numbers.add(100))           // Dangerous
    .forEach(System.out::println);
```

May cause:

- `ConcurrentModificationException`
 - Unexpected behavior
-

✔ 8 Complete List of Accepted Input Types

For `peek()`:

- Lambda returning void
- Method reference returning void

- Static method reference
- Instance method reference
- Anonymous class
- Consumer variable

Primitive variants:

Method	Functional Interface
IntStream.peek()	IntConsumer
LongStream.peek()	LongConsumer
DoubleStream.peek()	DoubleConsumer

❑ Interview-Ready Answer

The `peek()` method accepts a single parameter of type `Consumer<T>`. It is primarily used for debugging inside a stream pipeline. The consumer must return void and can be implemented using a lambda, method reference, anonymous class, or Consumer variable. Primitive streams use specialized interfaces like `IntConsumer`. Since `peek()` is lazy, it executes only when a terminal operation is invoked.

If you'd like next:

- ❑ Internal execution flow of `peek()`
- ☼ Bytecode-level explanation
- ❑ Parallel stream behavior with `peek()`
- ❑ UML execution diagram
- ❑ `peek()` vs `forEach()` deep comparison
- ❑ Java 1.5 equivalent debugging approach

✔ Java Stream API `peek()` — Metrics Collection

❑ 1 Official Method Signature

`Stream<T> peek(Consumer<? super T> action)`

❑ `peek()` accepts **only ONE parameter**:

Stream Type	Accepted Functional Interface
Stream<T>	Consumer<T>
IntStream	IntConsumer
LongStream	LongConsumer
DoubleStream	DoubleConsumer

- ✓ The function must return **void**
- ✓ Used for logging, debugging, and **lightweight metrics tracking**

✓ 2 What “Metrics” Means in peek()

Common metrics collected using peek():

- Counting processed elements
- Timing execution
- Tracking min/max
- Counting filtered elements
- Recording statistics
- Monitoring pipeline stages

⚠ peek() should not contain heavy business logic.

✓ 3 All Input Forms Accepted by peek() for Metrics

1 Lambda Expression (Counting Metrics)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsExample1 {
    public static void main(String[] args) {

        AtomicInteger counter = new AtomicInteger();

        List<String> list = Arrays.asList("Java", "Stream", "API");

        list.stream()
```

```
        .peek(s -> counter.incrementAndGet())
        .forEach(System.out::println);

    System.out.println("Total processed: " + counter.get());
}
}
```

- ✓ Uses Consumer<String>
 - ✓ Thread-safe counter
-

2 Method Reference (Custom Metrics Method)

```
public static void track(String value) {
    System.out.println("Tracking: " + value);
}
```

```
list.stream()
    .peek(PeekMetricsExample1::track)
    .forEach(System.out::println);
```

- ✓ Method must return void
-

3 Consumer Variable (Reusable Metrics Collector)

```
import java.util.function.Consumer;

AtomicInteger count = new AtomicInteger();

Consumer<String> metrics = s -> count.incrementAndGet();

list.stream()
    .peek(metrics)
    .forEach(System.out::println);

System.out.println("Count: " + count.get());
```

4 Anonymous Class

```
list.stream()
    .peek(new java.util.function.Consumer<String>() {
        @Override
        public void accept(String s) {
```

```
        System.out.println("Metric captured: " + s);
    }
})
.forEach(System.out::println);
```

5 Timing Metrics Example

```
long start = System.nanoTime();

list.stream()
    .peek(s -> System.out.println("Processing: " + s))
    .forEach(System.out::println);

long end = System.nanoTime();
System.out.println("Time taken (ns): " + (end - start));
```

✔ 4 Metrics in Pipeline Stages

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsExample2 {
    public static void main(String[] args) {

        AtomicInteger filteredCount = new AtomicInteger();

        List<Integer> numbers = Arrays.asList(1,2,3,4,5);

        numbers.stream()
            .filter(n -> n % 2 == 0)
            .peek(n -> filteredCount.incrementAndGet())
            .map(n -> n * 10)
            .forEach(System.out::println);

        System.out.println("Even numbers count: " + filteredCount.get());
    }
}
```

✔ Counts filtered elements

✔ 5 Primitive Stream Metrics

IntStream Example

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.IntStream;

public class PeekPrimitiveMetrics {
    public static void main(String[] args) {

        AtomicInteger counter = new AtomicInteger();

        IntStream.range(1,6)
            .peek(n -> counter.incrementAndGet())
            .map(n -> n * 2)
            .forEach(System.out::println);

        System.out.println("Total processed: " + counter.get());
    }
}
```

✓ Uses IntConsumer

✓ 6 Using Logging Framework for Metrics

```
import java.util.logging.Logger;

Logger logger = Logger.getLogger("MetricsLogger");

list.stream()
    .peek(logger::info)
    .forEach(System.out::println);
```

✓ Logger method must return void

✓ 7 Important Rules for Metrics with peek()

✓ peek() is:

- Intermediate operation
- Lazy (needs terminal operation)
- Safe for lightweight metrics

⚠ For parallel streams:

- Use thread-safe structures (AtomicInteger)
- Avoid mutable shared state without synchronization

✗ Not Recommended

```
.peek(s -> database.save(s));
```

```
// Heavy side effect ✗
```

Use `forEach()` for terminal business logic.

✓ 8 Complete List of Accepted Input Types

For `peek()`:

- Lambda returning void
- Method reference returning void
- Static method reference
- Instance method reference
- Anonymous class
- Consumer variable

Primitive variants:

Method	Functional Interface
<code>IntStream.peek()</code>	<code>IntConsumer</code>
<code>LongStream.peek()</code>	<code>LongConsumer</code>
<code>DoubleStream.peek()</code>	<code>DoubleConsumer</code>

☐ Interview-Ready Answer

The `peek()` method accepts a single parameter of type `Consumer<T>`. It is commonly used for debugging and lightweight metrics collection inside a stream pipeline. The consumer must return void and can be implemented as a lambda, method reference, anonymous class, or Consumer variable. Primitive streams use specialized interfaces like `IntConsumer`. Because `peek()` is lazy, it executes only when a terminal operation is invoked.

If you'd like next:

- ☐ Internal execution flow of `peek()`
- ⚙ Bytecode-level explanation
- ☐ Parallel stream metrics behavior

- ☐ UML execution diagram
- ☐ peek() vs forEach() deep comparison
- ☐ Java 1.5 equivalent metrics approach